TerrainScope

Off-Road Semantic Segmentation and Live Inference Dashboard

Team Name: Team Invincible
Team Members: Karna Bhosle, Alok Kushwaha, Shankar Gouda
Project Focus: Off-road semantic segmentation for terrain understanding
Tagline: A reproducible DINOv2 baseline with evaluation exports, failure analysis, and live review

---

Title & Summary

TerrainScope is an end-to-end off-road segmentation system. It predicts one of 10 classes per pixel, evaluates results with IoU, Dice, and pixel accuracy, exports review artifacts, and provides a frontend for live inference and scene analysis.

This submission was designed to be: - clear, - reproducible, - CPU-practical during development, - and straightforward for judges to review.

Final Headline Results

Split	Mean IoU	Mean Dice	Pixel Accuracy	Avg Loss
Validation	**30.30%**	**38.88%**	**75.35%**	**0.6690**
Test	**21.08%**	**26.27%**	**63.15%**	**1.2885**

Main Gain Over the Early Baseline

Stage	Validation IoU
Early quick baseline (`quick_cpu_50`)	6.03%
Final evaluated run (`quick_cpu_100x20`)	**30.30%**

The main gain came from fixing the label contract, stabilizing the pipeline, and extending training through resume support.

---

1. Problem

Off-road scenes are difficult to segment because: - terrain classes overlap visually, - vegetation and clutter blur boundaries, - small obstacle classes are easy to miss, - and Falcon masks use raw values instead of simple class IDs.

The system needed to solve four things: 1. predict 10 semantic classes per pixel, 2. keep the pipeline reproducible, 3. export strong evaluation evidence, 4. support live visual inspection.

Dataset Summary

Split	Images	Path
Train	2857	`Offroad_Segmentation_Training_Dataset/Offroad_Segmentation_Training_Dataset/train`

Split	Images	Path
Validation	317	`Offroad_Segmentation_Training_Dataset/Offroad_Segmentation_Training_Dataset/val`
Test	1002	`Offroad_Segmentation_testImages/Offroad_Segmentation_testImages`

10-Class Label Contract

Raw Value	Class
100	Trees
200	Lush Bushes
300	Dry Grass
500	Dry Bushes
550	Ground Clutter
600	Flowers
700	Logs
800	Rocks
7100	Landscape
10000	Sky

Critical engineering fix: - masks are read as raw `uint16`, - mapped through one strict shared label contract, - and unknown values are sent to `ignore_index = 255`.

Verified Raw Values by Split

Split Group	Raw Values Found
Train + Validation	`[100, 200, 300, 500, 550, 600, 700, 800, 7100, 10000]`
Test	`[100, 200, 300, 500, 800, 7100, 10000]`

Interpretation: - train and validation contain the full supported label contract, - test contains a valid subset of supported classes, - and no unsupported raw values were found in the verified scan.

---

2. Methodology

2.1 Training Pipeline

1. Load RGB images and Falcon masks from the actual nested dataset structure.
2. Read masks unchanged to preserve the original raw values.
3. Convert raw mask values into class IDs `0..9`.
4. Resize RGB images to `140 x 252` and normalize them.

5. Resize masks with nearest-neighbor interpolation.
6. Extract patch tokens from a frozen DINOv2 backbone.
7. Feed tokens into a lightweight segmentation head.
8. Upsample logits to full image size.
9. Train with cross-entropy loss using `ignore_index = 255`.
10. Save checkpoints, plots, comparisons, confusion matrices, and worst cases.

2.2 Architecture

```
Input RGB Image + Falcon Mask
        |
        v
Preprocessing
- resize
- normalize
- raw-to-class conversion
        |
        v
Frozen DINOv2 Backbone (dinov2_vits14)
        |
        v
Patch Tokens
        |
        v
ConvNeXt-style Segmentation Head
        |
        v
Upsampled 10-Class Prediction
        |
        +--> Metrics
        +--> Checkpoints
        +--> Evaluation exports
        +--> Live API + React dashboard
```

2.3 Model and Setup

Item	Choice
Backbone	`dinov2_vits14`
Backbone mode	Frozen
Segmentation head	Lightweight ConvNeXt-style custom head
Input size	`140 x 252`
Patch size	`14`
Batch size	`1`
Optimizer	SGD
Learning rate	`0.0001`
Momentum	`0.9`
Total recorded epochs	`40`

Why this setup worked: - DINOv2 provided strong pretrained visual features. - Freezing the backbone made CPU training feasible. - The lightweight head kept iteration time manageable.

2.4 Preprocessing and Data Handling Details

Step	Choice	Reason
RGB resize	Bilinear interpolation	Preserves image quality
Mask resize	Nearest-neighbor interpolation	Prevents label mixing
RGB normalization mean	`[0.485, 0.456, 0.406]`	Standard pretrained input scaling
RGB normalization std	`[0.229, 0.224, 0.225]`	Standard pretrained input scaling
Mask read mode	`cv2.IMREAD_UNCHANGED`	Preserves Falcon raw values
Ignore index	`255`	Handles unsupported or masked pixels safely

2.5 Segmentation Head Design

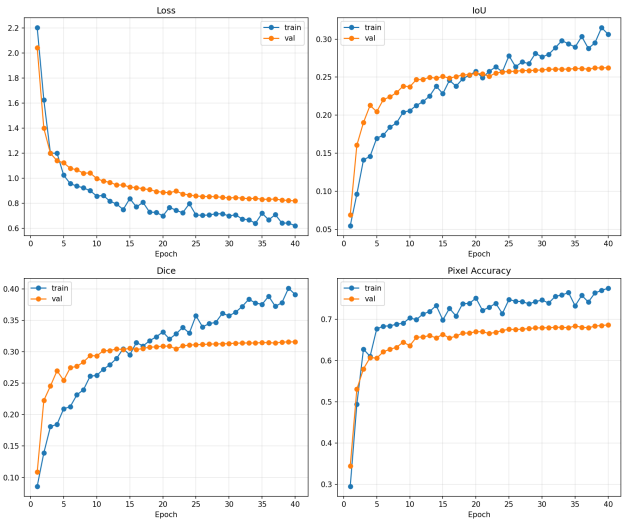
The segmentation head is a lightweight ConvNeXt-style module trained on top of frozen DINOv2 patch tokens.

Head structure: - 7x7 convolution stem to 128 channels - GELU activation - depthwise 7x7 convolution block - GELU activation - pointwise 1x1 convolution - GELU activation - final 1x1 classifier to 10 output classes

Why this head was chosen: - simple enough for CPU training, - stronger than a single linear classifier, - and suitable for a reproducible baseline built around frozen pretrained features.

2.6 Training Trend

The chart below shows loss, IoU, Dice, and pixel-accuracy trends across the run.



Training Metrics

---

3. Results & Performance Metrics

3.1 Overall Results

Metric	Validation	Test
Mean IoU	**30.30%**	**21.08%**
Mean Dice	**38.88%**	**26.27%**
Pixel Accuracy	**75.35%**	**63.15%**
Average Loss	**0.6690**	**1.2885**

3.2 Training-Time Validation vs Full Validation

The training loop used capped validation batches for faster feedback. The standalone evaluation pass used the full validation set and is the correct number to report publicly.

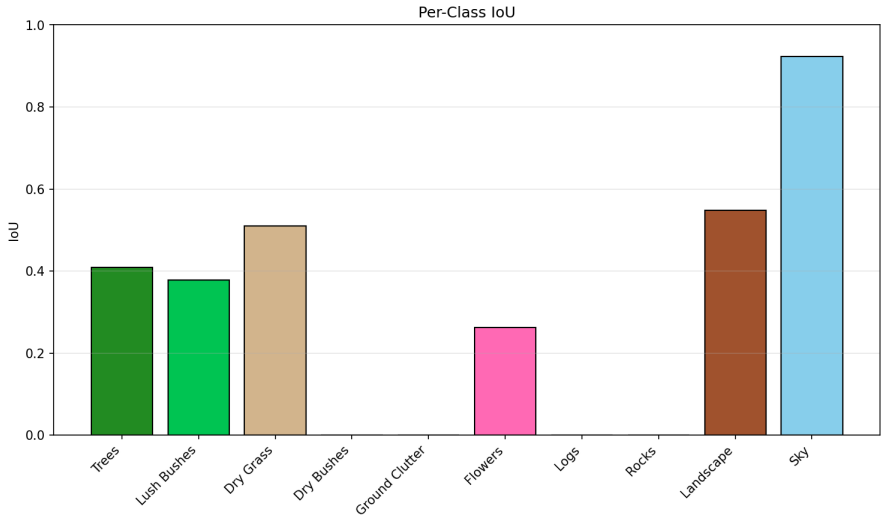
Validation View	Mean IoU	Mean Dice	Pixel Accuracy	Notes
Training-time summary	26.24%	31.59%	68.61%	Faster, capped batches
Full validation evaluation	**30.30%**	**38.88%**	**75.35%**	Final report metric

3.3 Per-Class Validation Summary

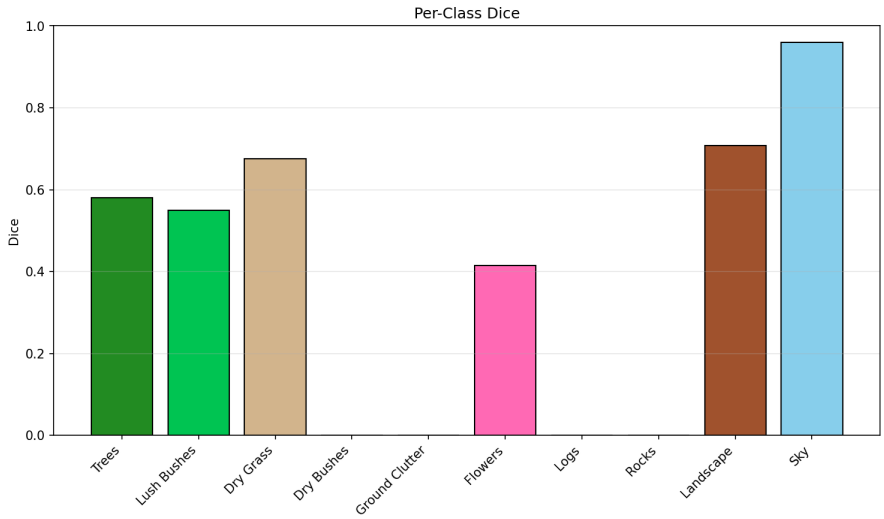
Class	IoU	Dice	Observation
Trees	40.84%	57.99%	Good
Lush Bushes	37.87%	54.94%	Good
Dry Grass	51.06%	67.60%	Strong
Dry Bushes	0.00%	0.00%	Very weak
Ground Clutter	0.03%	0.07%	Very weak
Flowers	26.19%	41.51%	Moderate
Logs	0.00%	0.00%	Very weak
Rocks	0.00%	0.00%	Very weak
Landscape	54.77%	70.77%	Strong
Sky	92.24%	95.96%	Very strong

Key reading: - The model performs best on large and visually stable classes such as **Sky**, **Landscape**, and **Dry Grass**. - Smaller or rarer classes remain difficult, especially **Logs**, **Rocks**, **Ground Clutter**, and **Dry Bushes**.

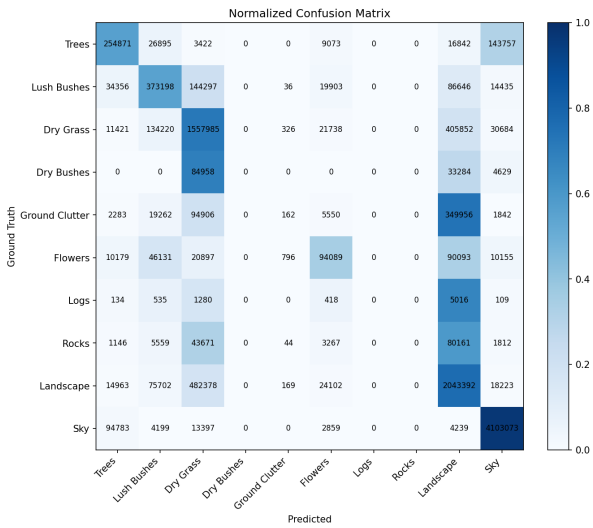
3.4 Required Charts



Per-Class IoU



Per-Class Dice



Confusion Matrix

3.5 Test-Split Interpretation

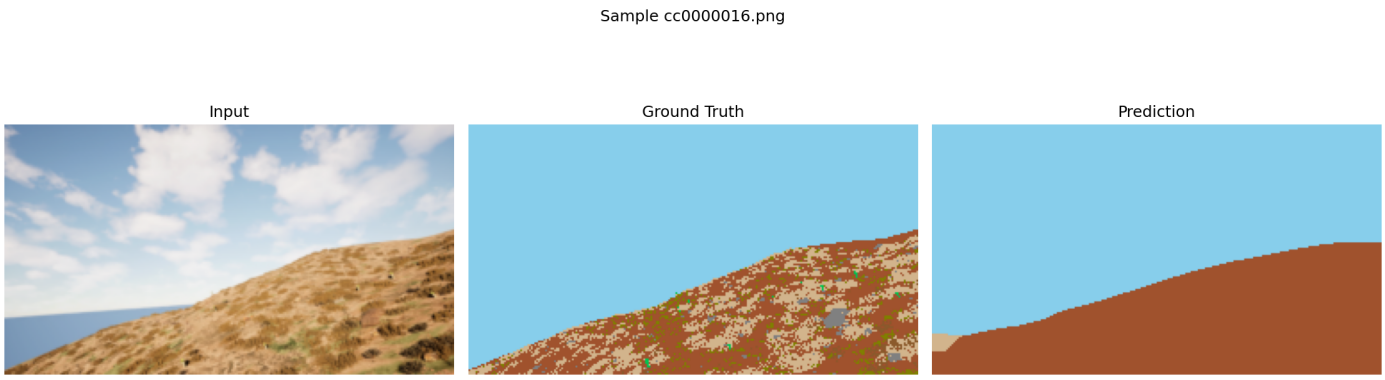
The test split is harder than validation and shows the main generalization gap:

Metric	Validation	Test	Gap
Mean IoU	30.30%	21.08%	-9.22 pts
Mean Dice	38.88%	26.27%	-12.61 pts
Pixel Accuracy	75.35%	63.15%	-12.20 pts

This gap suggests: - the model generalizes well on broad scene structure, - but still struggles when class frequency or scene composition changes, - especially for rare and clutter-heavy categories.

3.6 Representative Prediction Example

The figure below shows a representative validation comparison from the saved run artifacts.



Representative Validation Comparison

Interpretation: - large background regions are segmented more reliably than small clutter classes, - broad scene structure is captured well, - rare obstacle classes still drive most of the remaining error.

---

4. Challenges & Solutions

Challenge	What Went Wrong	Solution Applied	Result
Raw Falcon masks	Labels could be corrupted by naive reading	Read masks unchanged and enforce a strict 10-class mapping	Stable labels and training
Dataset path mismatch	Scripts did not match the real folder structure	Standardized the true nested train/val/test paths	Reproducible loading
CPU training cost	Heavy models were too slow locally	Frozen DINOv2 + lightweight head + quick CPU config	Feasible local training
Long runs were hard to finish	One short run was not enough	Added checkpoint resume support	Reached 40 recorded epochs
Weak presentation quality	Static metrics alone were not enough	Added evaluation exports, API backend, and React frontend	Better demo and review flow

Example Entry

1 **Task:** Model training on dataset

1 **Initial IoU Score:** 6.03%

1 **Issue Faced:** Early performance was too weak, especially on rare classes.

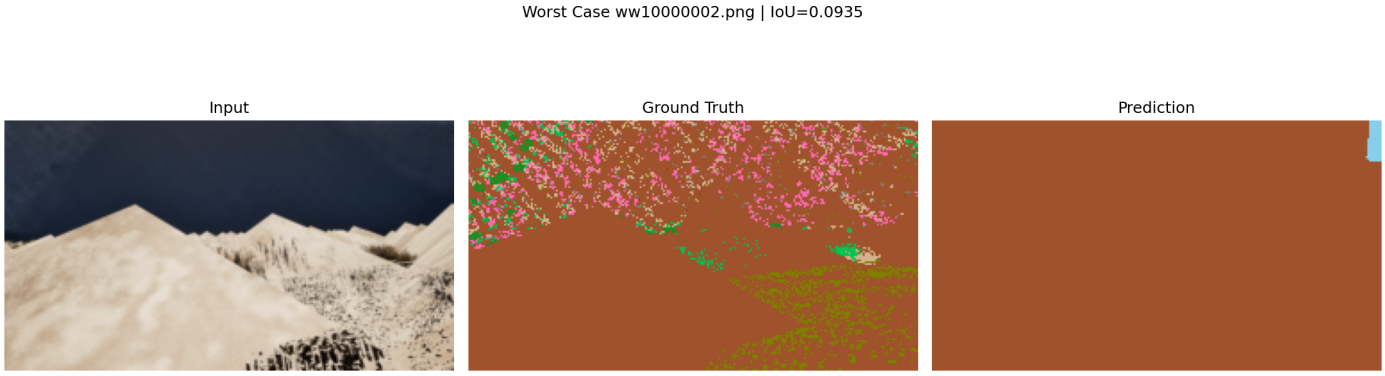
1 **Solution:** Fixed the data contract, improved pipeline stability, and trained longer with resume support.

1 **New Score:** Validation IoU improved to **30.30%**.

4.1 What Was Improved in the Engineering Stack

Beyond model training, the project was upgraded into a full submission kit: - shared dataset, label, metric, and reporting utilities under `offroad_segmentation`, - config-driven `train.py` and `test.py`, - checkpoint resume support, - visual artifact export, - a FastAPI live inference backend, - and a React dashboard for run browsing and upload analysis.

Failure Cases

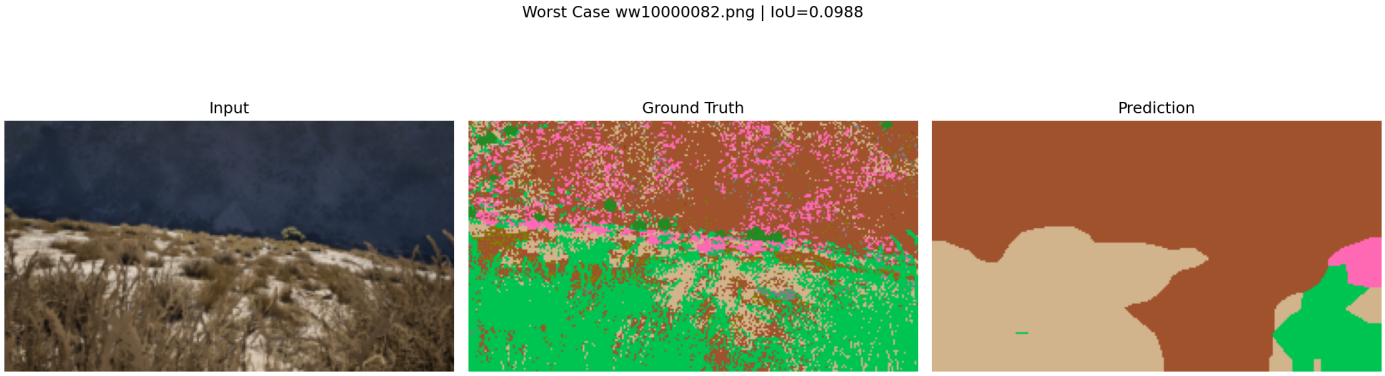


Failure Case 1

What happened:
The model struggled in a complex scene with mixed terrain and fine-grained regions.

Why it likely happened:
Rare and small classes receive limited supervision in the current CPU-friendly setup.

Potential fix:
Use class-balanced loss, stronger augmentation, and a richer decoder.



Failure Case 2

What happened:
Class boundaries were unstable in cluttered terrain.

Why it likely happened:
Low-resolution training and frozen features reduce sensitivity to small details.

Potential fix:
Use higher-resolution training on GPU and partial backbone fine-tuning.

---

5. Conclusion & Future Work

5.1 Deployment and Demonstration Layer

The project includes a usable demo layer in addition to offline evaluation.

Backend: - FastAPI server for run discovery, asset serving, and upload inference

Frontend: - React dashboard for: - metric review, - split switching, - comparison view, - fullscreen analysis, - live upload inference, - and terrain suggestions beside the image

This improves the submission quality because judges can review both: - quantitative performance, - and qualitative model behavior on real scenes.

Conclusion

TerrainScope delivered a complete and reproducible segmentation kit with: - corrected label handling, - train and test pipelines, - validation and test evaluation, - full artifact export, - and a live dashboard for browsing runs and uploading images.

The final system is strongest on: - Sky - Landscape - Dry Grass

The main performance gap remains in: - Logs - Rocks - Ground Clutter - Dry Bushes

Final headline metrics: - Validation Mean IoU: **30.30%** - Validation Mean Dice: **38.88%** - Validation Pixel Accuracy: **75.35%** - Test Mean IoU: **21.08%** - Test Mean Dice: **26.27%** - Test Pixel Accuracy: **63.15%**

Future Work

1. Train on larger coverage and higher resolution with GPU support.
2. Add class-balanced loss or sampling for rare classes.
3. Use a stronger segmentation decoder.
4. Fine-tune upper DINOv2 layers instead of freezing the full backbone.
5. Add targeted augmentation for clutter, occlusion, and obstacle-heavy scenes.

---

6. Reproducibility and Submission Files

6.1 Key Output Artifacts

Important generated files for the final run: - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/checkpoints/best_iou.pth` - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/plots/training_metrics.png` - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/evaluation_metrics.txt` - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/per_class_iou.png` - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/confusion_matrix.png` - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/comparisons/` - `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/evaluations/val/worst_cases/`

Reproduction Commands

```
python -m pip install -r .\Offroad_Segmentation_Scripts\requirements.txt
python .\Offroad_Segmentation_Scripts\train.py --run_name dinov2_baseline
python .\Offroad_Segmentation_Scripts\test.py --model_path .\Offroad_Segmentation_Scripts\runs\dinov2_baseli
python .\Offroad_Segmentation_Scripts\test.py --model_path .\Offroad_Segmentation_Scripts\runs\dinov2_baseli
```

Submission Checklist

1. `Offroad_Segmentation_Scripts/train.py`
1. `Offroad_Segmentation_Scripts/test.py`
1. `Offroad_Segmentation_Scripts/configs/`
1. `Offroad_Segmentation_Scripts/offroad_segmentation/`
1. `Offroad_Segmentation_Scripts/runs/quick_cpu_100x20/`
1. `README.md`
1. `project_documentation.txt`
1. `report.md`
1. `final_report.md`

This report is the polished markdown version with percentage-based metrics and embedded charts for export.